

SAND — Secure Archival Network Distribution

Architecture Document v1.0

1. Project Overview

SAND is a file archival tool that provides **security through splitting and encryption**. Given any file, SAND compresses it, splits it into two equal parts, generates a third redundancy part via XOR, encrypts all three with AES-256-GCM using an Argon2id-derived key, and outputs three `.media` files. Any two of the three parts are sufficient to reconstruct the original file.

The tool ships as a single static Go binary with two operating modes:

- **CLI mode** — command-line archive and restore operations
 - **Server mode** — a local HTTP server with an embedded React web GUI
-

2. Cryptographic Design

2.1 Key Derivation — Argon2id

The user-supplied password is stretched into a 256-bit AES key using Argon2id.

Parameter	Value	Rationale
Time	3 iterations	Balance between security and UX latency
Memory	64 MB	Resistant to GPU/ASIC attacks
Threads	4	Parallelism for modern CPUs
Salt	16 bytes	Cryptographically random, unique per operation
Key length	32 bytes	AES-256

A single salt is generated per archive operation. All three parts share the same derived key but use **unique nonces**. The salt and Argon2id parameters are stored in each media file's header so the key can be re-derived during restore.

2.2 Encryption — AES-256-GCM

Each part is encrypted independently with AES-256-GCM:

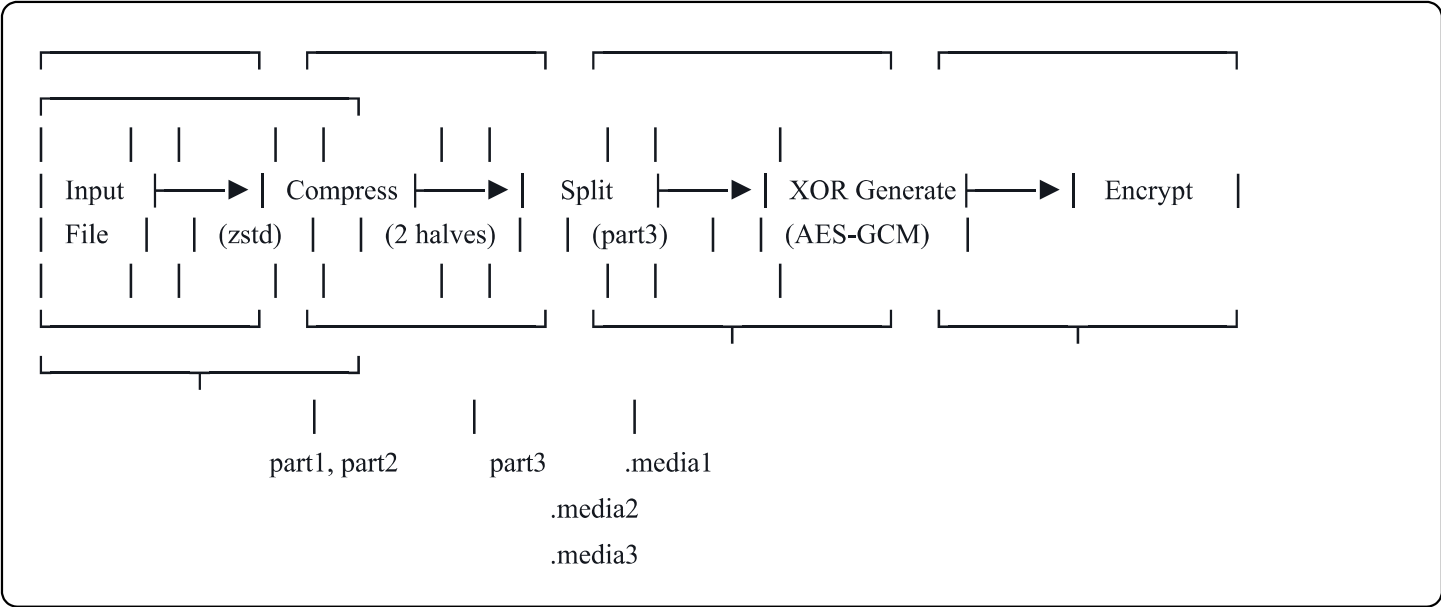
- **Nonce:** 12 bytes, cryptographically random, unique per part
- **Associated Data (AD):** the media file header (part number, version, original filename hash) — binds ciphertext to its metadata and prevents part-swapping attacks
- **Tag:** 16 bytes (GCM standard), appended to ciphertext

2.3 Integrity Guarantees

Layer	Mechanism	Protects Against
Per-part	AES-GCM authentication tag	Tampering, bit-flipping, truncation
Original file	SHA-256 hash in header	Verifies final reconstruction
Header binding	GCM Associated Data	Part-swapping, metadata tampering

3. Data Pipeline

3.1 Archive Pipeline

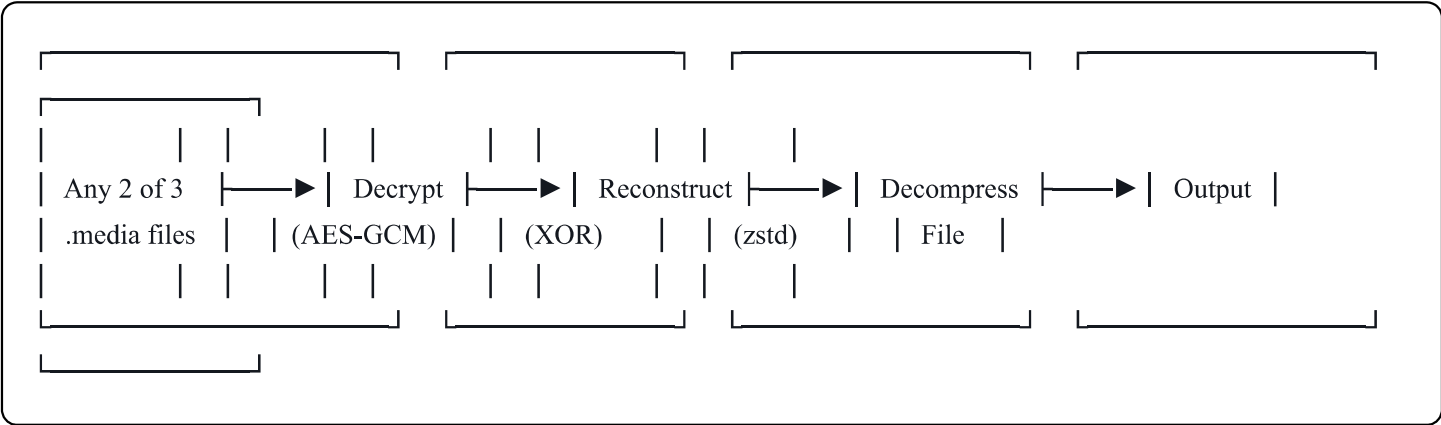


Steps in detail:

1. **Read** the input file into memory (streaming for large files — see §6)
2. **Hash** the original file with SHA-256 (stored in headers for verification)
3. **Compress** the file contents with zstd (default compression level 3)
4. **Pad** the compressed data to even length by appending one `0x00` byte if the byte count is odd

- 5. **Split** the (possibly padded) compressed data into two equal halves: `part1` and `part2`
- 6. **XOR** `part1` and `part2` byte-by-byte to produce `part3`
- 7. **Derive** AES-256 key from password + random salt via Argon2id
- 8. **Encrypt** each part independently (unique nonce per part) with AES-256-GCM
- 9. **Write** three output files: `<filename>.media1`, `<filename>.media2`, `<filename>.media3`

3.2 Restore Pipeline



Steps in detail:

- 1. **Read** headers from the two provided media files
- 2. **Validate** that the two parts are from the same archive (matching archive ID, version)
- 3. **Derive** AES-256 key from password + salt (salt read from header)
- 4. **Decrypt** both parts using AES-256-GCM (GCM tag verification catches wrong passwords or corruption)
- 5. **Reconstruct** the missing part:
 - If `part3` is missing: no reconstruction needed — concatenate `part1` + `part2`
 - If `part2` is missing: `part2 = XOR(part1, part3)`
 - If `part1` is missing: `part1 = XOR(part2, part3)`
- 6. **Concatenate** `part1` + `part2` to get the padded compressed data
- 7. **Strip padding** if the original had odd byte count (flag stored in header)
- 8. **Decompress** with `zstd`
- 9. **Verify** SHA-256 hash matches the original file hash stored in headers
- 10. **Write** the restored file

3.3 Reconstruction Truth Table

Have part1	Have part2	Have part3	Recovery Method
✓	✓	✗	Direct: concat(part1, part2)

Have part1	Have part2	Have part3	Recovery Method
✓	✗	✓	part2 = XOR(part1, part3), concat
✗	✓	✓	part1 = XOR(part2, part3), concat
✓	✓	✓	Direct: concat(part1, part2)
✗	✗	✓/✗	✗ Unrecoverable

4. Media File Format

Each `.media{N}` file has a binary header followed by encrypted payload.

4.1 Header Layout (Binary, Big-Endian)

Offset	Size	Field	Description
0x00	4	Magic	"SAND" (0x53414E44)
0x04	1	Version	Format version (currently 1)
0x05	1	PartNumber	1, 2, or 3
0x06	16	ArchiveID	Random UUID — ties parts together
0x16	32	OriginalHash	SHA-256 of the original file
0x36	8	OriginalSize	Uncompressed file size (uint64)
0x3E	8	CompressedSize	After zstd, before split (uint64)
0x46	1	WasPadded	1 if compressed data was odd-length
0x47	2	FilenameLength	Length of original filename (uint16)
0x49	var	Filename	Original filename (UTF-8, max 512 bytes)
var	16	Salt	Argon2id salt
var	4	Argon2Time	Argon2id time parameter
var	4	Argon2Memory	Argon2id memory parameter (KB)
var	1	Argon2Threads	Argon2id parallelism
var	12	Nonce	AES-GCM nonce (unique per part)
var	4	PayloadSize	Encrypted payload size (uint32)
var	N	Payload	AES-GCM ciphertext + 16-byte auth tag

4.2 Design Decisions

- **ArchiveID** (UUID v4): Lets the tool verify that two parts belong to the same archive before attempting decryption.
- **Redundant headers**: Each media file is self-describing. Metadata is duplicated across all three files so that any two files contain everything needed for restoration.

- **Filename stored in header:** Enables the restore operation to name the output file correctly without user input.
 - **Header as GCM Associated Data:** The header bytes (up to but not including the Payload) are passed as Associated Data to AES-GCM, cryptographically binding the metadata to the ciphertext.
-

5. Project Structure

```
sand/
├── cmd/
│   └── sand/
│       └── main.go          # Entry point, CLI parsing
├── internal/
│   ├── archive/
│   │   ├── archive.go      # Archive pipeline orchestrator
│   │   ├── restore.go      # Restore pipeline orchestrator
│   │   └── archive_test.go
│   ├── crypto/
│   │   ├── argon2.go       # Key derivation (Argon2id)
│   │   ├── aes.go          # AES-256-GCM encrypt/decrypt
│   │   └── crypto_test.go
│   ├── compress/
│   │   ├── zstd.go         # Zstd compress/decompress wrapper
│   │   └── compress_test.go
│   ├── splitter/
│   │   ├── split.go        # Split, XOR, reconstruct logic
│   │   └── split_test.go
│   ├── mediafile/
│   │   ├── header.go       # Header serialization/deserialization
│   │   ├── writer.go       # Write .media files
│   │   ├── reader.go       # Read .media files
│   │   └── mediafile_test.go
│   └── server/
│       ├── server.go       # HTTP server, API routes
│       ├── handlers.go     # Request handlers
│       └── server_test.go
├── web/                    # React frontend (embedded at build)
│   └── src/
│       ├── App.jsx
│       ├── components/
│       │   ├── ArchiveForm.jsx # Upload file + password → archive
│       │   └── RestoreForm.jsx  # Upload 2+ parts + password → restore
```

```

| | | └─ ProgressBar.jsx
| | | └─ FileDownload.jsx
| | └─ index.jsx
| └─ package.json
| └─ vite.config.js
└─ go.mod
└─ go.sum
└─ Makefile
└─ README.md

```

6. Module Responsibilities

6.1 `cmd/sand` — CLI Interface

Uses `cobra` for command-line parsing.

```

sand archive <file> [--password <pw>] [--output-dir <dir>]
sand restore <file> --parts <file1>,<file2> [--password <pw>] [--output-dir <dir>]
sand serve [--port 8080] [--bind 127.0.0.1]

```

- If `--password` is omitted, the CLI prompts interactively (stdin, no echo).
- `archive` produces three `.media` files in the output directory.
- `restore` accepts 2 or 3 `.media` files and reconstructs the original.
- `serve` starts the embedded web server.

6.2 `internal/crypto` — Cryptographic Operations

```

go

// DeriveKey generates a 32-byte AES key from password + salt using Argon2id.
func DeriveKey(password string, salt []byte, params Argon2Params) []byte

// Encrypt encrypts plaintext with AES-256-GCM using the given key.
// Returns (nonce || ciphertext || tag). Nonce is randomly generated.
func Encrypt(key, plaintext, associatedData []byte) ([]byte, error)

// Decrypt decrypts AES-256-GCM ciphertext.
// Expects input as (nonce || ciphertext || tag).
func Decrypt(key, ciphertext, associatedData []byte) ([]byte, error)

```

6.3 `internal/compress` — Compression

```
go

func Compress(data []byte) ([]byte, error) // zstd level 3
func Decompress(data []byte) ([]byte, error)
```

6.4 `internal/splitter` — Split, XOR, Reconstruct

```
go

// Split pads (if needed) and splits data into two equal halves.
func Split(data []byte) (part1, part2 []byte, wasPadded bool)

// XOR produces the redundancy part: result[i] = a[i] ^ b[i].
func XOR(a, b []byte) []byte

// Reconstruct returns the original data from any two of three parts.
func Reconstruct(parts map[int][]byte, wasPadded bool) ([]byte, error)
```

6.5 `internal/mediafile` — File Format

Handles serialization/deserialization of the binary media file format defined in §4.

6.6 `internal/archive` — Pipeline Orchestration

Coordinates the full archive and restore pipelines:

```
go

func Archive(inputPath, password, outputDir string) error
func Restore(mediaPaths []string, password, outputDir string) error
```

6.7 `internal/server` — HTTP Server + Embedded Frontend

Embeds the production-built React app using Go's `embed` package.

7. HTTP API

The web server exposes a simple REST API. All endpoints are local-only by default.

7.1 Endpoints

Method	Path	Description
GET	/	Serve React SPA
POST	/api/archive	Upload file + password → get 3 parts
POST	/api/restore	Upload 2-3 parts + password → get file
GET	/api/health	Health check

7.2 Archive Request

POST /api/archive
Content-Type: multipart/form-data

Fields:

- file: <binary file>
- password: <string>

Response: A ZIP archive containing the three `.media` files, streamed as download.

200 OK
Content-Type: application/zip
Content-Disposition: attachment; filename="<original>.sand.zip"

7.3 Restore Request

POST /api/restore
Content-Type: multipart/form-data

Fields:

- parts[]: <media file 1>
- parts[]: <media file 2>
- password: <string>

Response: The reconstructed original file, streamed as download.

200 OK
Content-Type: application/octet-stream
Content-Disposition: attachment; filename="<original filename from header>"

7.4 Error Responses

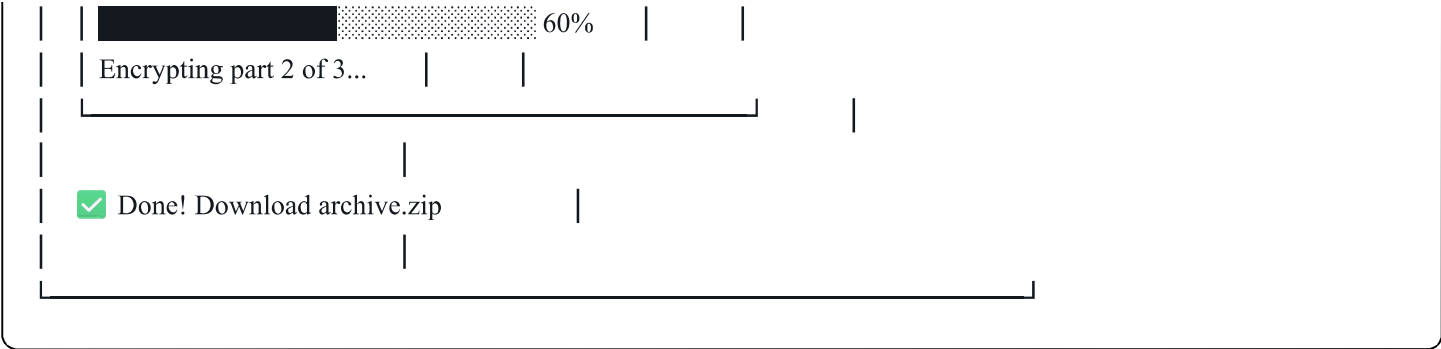
```
json
{
  "error": "description of what went wrong",
  "code": "WRONG_PASSWORD | CORRUPT_FILE | MISMATCHED_PARTS | ..."
}
```

8. Web GUI Design

The React frontend is minimal, intuitive, and functional.

8.1 Layout



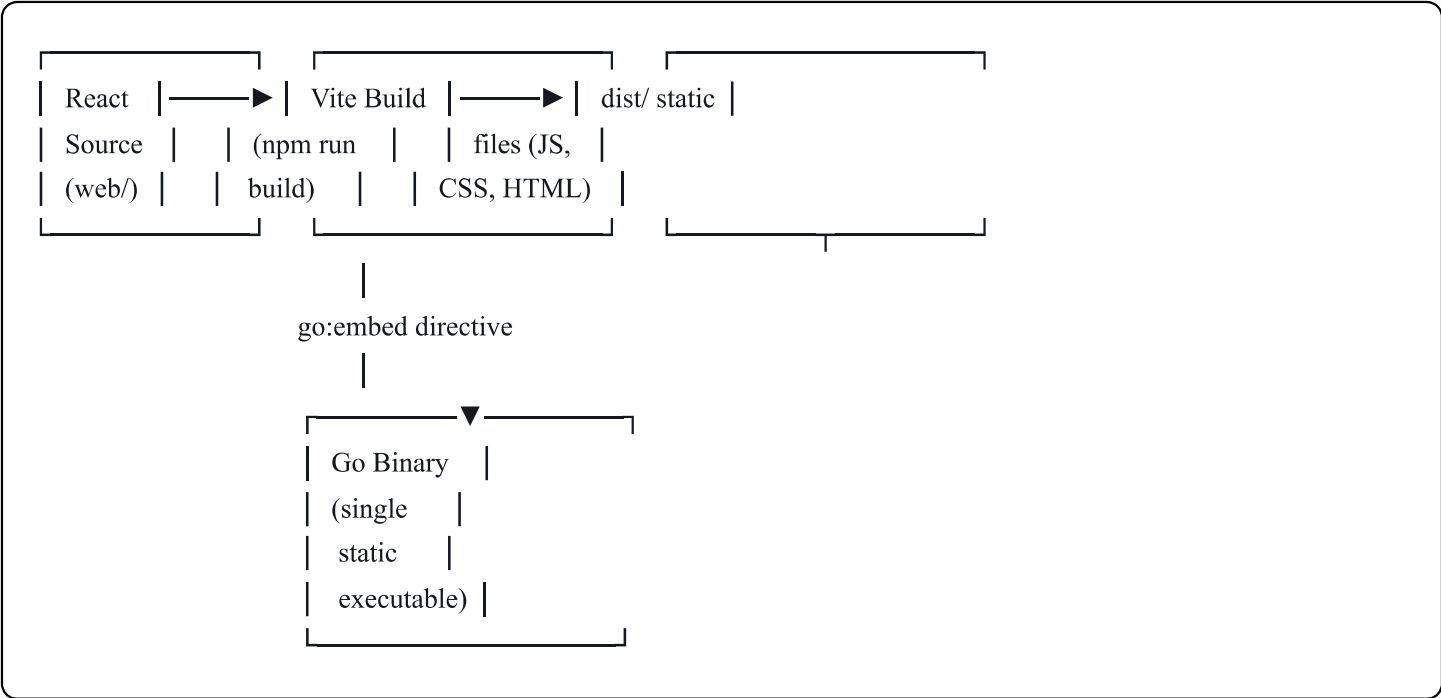


8.2 Key UX Decisions

- **Two tabs:** Archive and Restore — the two primary actions.
 - **Drag-and-drop** file upload with click fallback.
 - **Restore tab** accepts 2 or 3 `.media` files (shows which parts are detected).
 - **Password field** with show/hide toggle; no confirmation field (we verify via GCM tag).
 - **Progress feedback** during processing (compression, encryption stages).
 - **Auto-download** of result when complete.
-

9. Build & Embedding Strategy

9.1 Build Pipeline



9.2 Makefile Targets

```
makefile
```

```
build:      # Build React frontend, then Go binary
build-web:  # npm install && npm run build (in web/)
build-go:   # CGO_ENABLED=0 go build -o sand ./cmd/sand
test:      # Run Go tests
clean:     # Remove build artifacts
```

9.3 Embedding

```
go

//go:embed web/dist/*
var webAssets embed.FS
```

The embedded filesystem is served by the HTTP server in `serve` mode. This keeps the final deliverable as a single static binary.

10. Go Dependencies

Dependency	Purpose
<code>golang.org/x/crypto/argon2</code>	Argon2id key derivation
<code>github.com/klauspost/compress/zstd</code>	Zstd compression
<code>github.com/spf13/cobra</code>	CLI framework
<code>github.com/google/uuid</code>	Archive ID generation
<code>crypto/aes</code> , <code>crypto/cipher</code> (stdlib)	AES-256-GCM encryption
<code>crypto/rand</code> , <code>crypto/sha256</code> (stdlib)	Random generation, hashing
<code>embed</code> , <code>net/http</code> (stdlib)	Static file embedding, HTTP

Note: `klauspost/compress` is a pure Go implementation — no CGO required. This preserves fully static binary builds with `CGO_ENABLED=0`.

11. Security Considerations

11.1 Threat Model

Threat	Mitigation
Brute-force password attack	Argon2id with 64MB memory cost
Ciphertext tampering	AES-GCM authentication tag on every part
Part-swapping attack	Header bound as GCM Associated Data
Single cloud compromise	Attacker gets at most 1 of 3 parts — cannot reconstruct
Metadata leakage	Filename is inside encrypted header binding
Password in CLI history	Interactive prompt (no echo) when --password is omitted
Memory exposure	Zero password/key bytes after use

11.2 What SAND Does NOT Protect Against

- **Keyloggers / endpoint compromise:** If the machine running SAND is compromised, the password and plaintext are exposed.
- **Weak passwords:** Argon2id slows brute-force but cannot save a 4-character password.
- **Two-provider compromise:** If an attacker obtains any 2 of 3 media files AND the password, the original file is recoverable. SAND's distribution model assumes independent storage providers.

12. Future Considerations (Out of Scope for v1)

These items are noted for architectural awareness but will not be built in the initial version:

- **Streaming / chunked processing** for files larger than available RAM
- **Cloud provider integration** (Google Drive, Dropbox, iCloud APIs)
- **Multiple file / directory archival**
- **Configurable split count** (N parts with M redundancy, Reed-Solomon)
- **Key file support** (in addition to or instead of passwords)
- **Progress via WebSocket** (currently REST polling is sufficient)

13. Summary

SAND v1 delivers a single Go binary that:

1. **Archives** any file into 3 encrypted parts with XOR redundancy
2. **Restores** the original from any 2 of 3 parts
3. Operates via **CLI** or **Web GUI** (React, embedded in binary)
4. Uses **zstd** compression, **Argon2id** key derivation, and **AES-256-GCM** encryption
5. Builds as a fully **static binary** (no CGO, no runtime dependencies)